BUILDING LLM-POWERED APPLICATIONS · LECTURE 2

Structured Output & JSON

Turn free-form text into data your code can trust.

📋 IN CLASS — DO THIS FIRST

Open today's form

Scan the code, or open the link on the course page

Press F for fullscreen · navigate with ← →

Why this matters

From chat to software

- An app needs fields, not paragraphs
- Code calls `data["price"]`, not "read the reply"
- One flaky parse breaks the whole pipeline

Today

By the end you can:

1. say why raw text is the wrong contract
2. prompt for JSON (and know what JSON mode / schema would add)
3. validate with `json.loads` + a check
4. repair by feeding the error back

The model is an untrusted input

Treat a model reply like a web form submitted by a stranger. You'd never run form data; don't trust raw model output either.

- It's non-deterministic text — the wrong contract between two pieces of software.
- Parse it defensively: a format it commits to, plus a parser that says yes/no.

Why not `eval`?

`eval(reply)` on model output is a remote-code-execution hole.

`ast.literal_eval` is safer but accepts *Python* literals (`'single'`, `True`, `None`) — not JSON. It hides bugs. Use json.loads.

An LLM is asked for a price and replies: Sure! The price is {'price': 19.99}. You call json.loads() on the whole reply. What happens?

- Returns {'price': 19.99} — it's a dict
- Returns 19.99 as a float
- **Raises JSONDecodeError — there's prose around it AND single quotes ✓**
- Returns None and prints a warning

"Just parse the text" fails

Ask for a price and the model is helpful in all the wrong ways:

```
# you wanted:  19.99
"The price is $19.99."           # prose around the number
"Sure! Here's the JSON:\n```json\n{...}\n```"  # markdown fences
"{'price': 19.99}"               # single quotes — not JSON
"{\"price\": 19.99,}"             # trailing comma
```

Regex and `str.split` work until the day they don't. We need a format the model commits to and a parser that says yes or no.

The model replies {'price': 19.99} (single quotes). A teammate suggests ast.literal_eval instead of json.loads. What's the real problem with that?

- literal_eval raises on single quotes, so it parses nothing
- **literal_eval happily accepts Python literals like single quotes and True, hiding bugs your real JSON clients would reject ✓**
- literal_eval is a remote-code-execution hole just like eval
- Nothing — literal_eval is the recommended way to parse model output

Prompting for JSON

The cheapest fix: ask precisely and give an example of the exact shape you want.

```
{"role": "system", "content": (
  "Extract the order. Reply with ONLY a JSON object, "
  "no markdown, no prose. Schema:\n"
  '{"item": str, "qty": int, "price": float}'
)}
```

- "ONLY JSON, no markdown" kills the ``` fences.
- Show one concrete example object (few-shot).
- Set temperature 0 — extraction should be repeatable.

JSON mode & response schemas

JSON mode

Some providers accept `response_format={"type": "json_object"}` and constrain decoding to valid JSON — but not the right *shape*.

Schema-constrained

Pass a JSON Schema (or `json_schema` mode) and the model is forced to emit exactly those keys and types.

Two different promises

Valid JSON ≠ valid data.

JSON mode stops syntax errors. A schema stops *missing fields* and *wrong types*. You usually want both — and you still validate on your side.

Heads-up: our class Gemma proxy IGNORES `response_format` (accepted, no effect). JSON mode is provider-specific (OpenAI etc.), not guaranteed on our model — so here the real mechanism is prompt → validate → repair.

Rank the guarantees: a good PROMPT, JSON MODE, and JSON SCHEMA. The model returns {"items": [...]} when you asked for {"item": ...}. Which is the WEAKEST layer that would have prevented this wrong-key reply?

- A good prompt — it shows the exact shape
- JSON mode — it constrains decoding
- **JSON Schema — it forces exactly those keys and types ✓**
- None of them — only your own validate() can catch wrong keys

No JSON mode here — so you are JSON mode

What the proxy does

Send `response_format` to our Gemma proxy and it is accepted and silently ignored. No constrained decoding, no syntax guarantee.

So the contract is yours

- Prompt for JSON + example
- Parse defensively (extract + repair)
- Validate keys, types, then values

Every guarantee a provider would give you, you reproduce in code. The next slides are the toolkit: a catalog of local repairs, a "which failure → which fix" routing table, and schemas past `{item, qty, price}`.

Schemas you don't hand-write: Pydantic

In real backends you define the shape once as a Pydantic model and reuse it everywhere.

```
from pydantic import BaseModel

class Order(BaseModel):
    item: str
    qty: int
    price: float

Order.model_json_schema()          # → JSON Schema for the model
Order.model_validate_json(text)    # → parse + validate
```

One class gives you the schema and the validator to check the reply. Pydantic is installed in our sandbox — it's the realistic backend tool (you'll use it in Lecture 3). Today's demos use stdlib `json` + a hand check so the validation logic stays fully visible.

You define one Pydantic Order(BaseModel) with item/qty/price. Before we run it: what does this ONE class give you that a hand-written validate() does not — for free?

- It forces gemma-4-E4B-it to emit valid JSON, so you can skip json.loads
- **Both a JSON Schema (model_json_schema) AND a parse+validate call (model_validate_json) with field-named errors ✓**
- Nothing extra — it's the same as isinstance checks, just slower
- It makes the proxy honor response_format so JSON mode finally works

Pydantic Demo

```
# Pydantic does parse + validate in ONE call — and gives a repair-ready error.
# Runs in the sandbox (pydantic is installed). No network needed. Ctrl/Cmd+Enter.
from pydantic import BaseModel, ValidationError


class Order(BaseModel):
    item: str
    qty: int
    price: float


good = '{"item": "mango", "qty": 3, "price": 45}'
bad = '{"item": "mango", "qty": "three", "price": 45}'    # qty is a string

for raw in (good, bad):
    try:
        order = Order.model_validate_json(raw)
```

THINK · PAIR · SHARE

Which layer catches which bug?

For each broken reply, name the weakest layer that would catch it — *prompt*, *JSON mode*, *JSON Schema*, or *your validate()*:

- `{"qty": 3,}` — trailing comma
- `{"item": "tea"}` — `price` missing
- `{"qty": "three"}` — wrong type
- `{"item": "tea", "qty": -3, "price": 25}`

2 min with your neighbour. Watch the last one — no provider feature catches it.

PREDICT → RUN → MODIFY

Before you run the next slide

We extract "3 mango smoothies, 45 baht each" into `{item, qty, price}` and print `qty * price`.

Predict two things:

- What total gets printed?
- Does `validate()` pass or reject?

Commit to a number, then run the next slide and check.

Parse Json

```python
# Ask the model for JSON, then parse it with the stdlib `json` module and a
# hand-written check. No pydantic here — just json.loads + validation.
import json
from openai import OpenAI

client = OpenAI()

SYSTEM = (
    "Extract the order from the user's message. "
    "Reply with ONLY a JSON object, no markdown and no prose. "
    'Schema: {"item": string, "qty": integer, "price": number}'
)

resp = client.chat.completions.create(
    model="gemma-4-E4B-it",
    temperature=0,
    messages=[
```

`extract_json`: a starting point, not the last word

It's deliberately simple, and first `{` ... last `}` can grab too much:

```
# two objects -> grabs across BOTH, parse fails
'{"a": 1} {"b": 2}'      # first { .. last } = '{"a": 1} {"b": 2}'
# a stray } in prose AFTER the json -> grabs too far
'{"a": 1}  see }?'
```

- Fine for these demos; in HW1 you harden it (auto-graded by `json.loads`-ing the result).
- The theme repeats: even your own parser treats the text as untrusted — start permissive, then tighten.

Valid JSON ≠ valid data

`{"item": "mango", "qty": -3, "price": -45}` parses and type-checks — yet a negative quantity is nonsense. Add semantic checks no schema can express:

```
if type(qty) is int and qty > 0: ...      # NOT isinstance — see below
if isinstance(price, (int, float)) and not isinstance(price, bool): ...
```

- The bool/int trap: `isinstance(True, int)` is True — bool subclasses int. Guard with `type(qty) is int`.
- List, not object: models return `[ {...} ]` sometimes — a leading `isinstance(data, dict)` rejects it cleanly.
- One number type: `45` is a valid "number" — accept `(int, float)` for price.

Your validate() uses `isinstance(qty, int)`. The model returns {"item": "tea", "qty": true, "price": 25}. What happens?

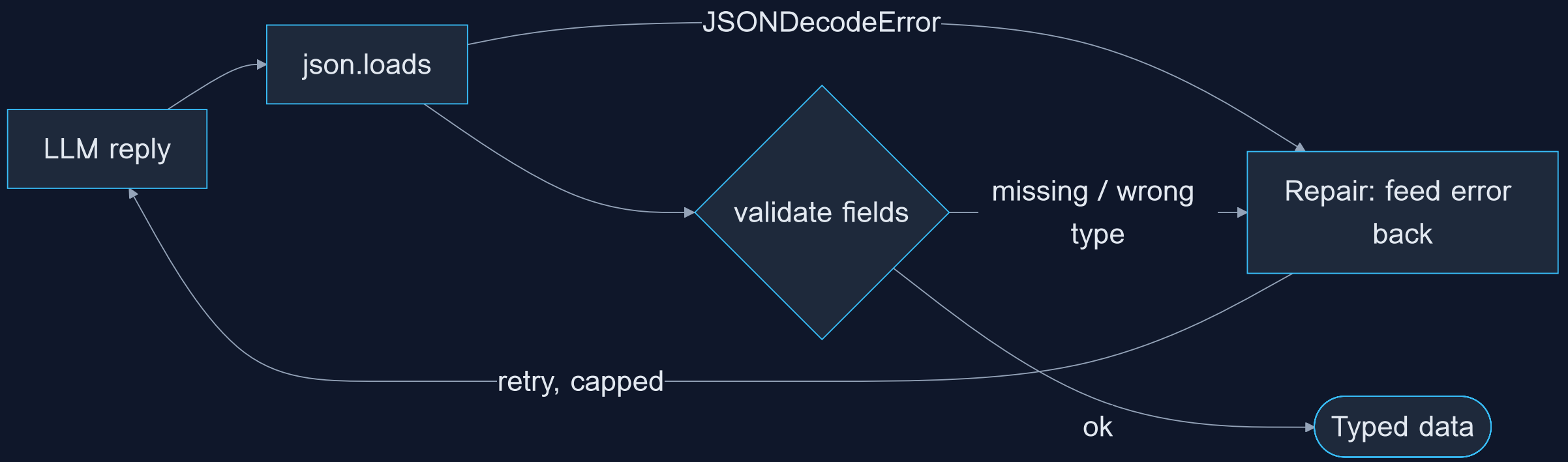
- Rejected — true is a bool, not an int
- **Accepted — in Python bool subclasses int, so isinstance(True, int) is True ✓**
- json.loads raises before validate even runs
- Accepted, and qty * price computes 25 because True == 1, so it's fine

Semantic Validation

```python
# Semantic validation: types alone don't make data CORRECT. No network needed —
# we feed fixed replies through the same check the model output would hit.
import json


def validate(data):
    if not isinstance(data, dict):            # rejects the [ {...} ] list case too
        raise ValueError("expected a JSON object")
    item, qty, price = data.get("item"), data.get("qty"), data.get("price")
    if not isinstance(item, str):
        raise ValueError("item must be a string")
    if type(qty) is not int or qty <= 0:      # type() not isinstance -> rejects bool
        raise ValueError("qty must be a positive integer")
    if not isinstance(price, (int, float)) or isinstance(price, bool) or price < 0:
        raise ValueError("price must be a non-negative number")
    return data
```

Validate Flow



"Repair" means two different things

Local repair

A known, mechanical malformation you fix in-process — no model call.

Classic case: a trailing comma. One regex, then re-parse. Cheap and instant.

Loop repair

The error is semantic or structural — you need the model's help.

Re-prompt with the error. Costs a round-trip (tokens, latency).

Always try the cheap local fix first; fall back to a model round-trip only when the string is broken in a way you can't safely patch. HW1 asks for both.

The repair loop

When validation fails, don't give up — feed the error back to the model and ask it to fix its own output.

```
for attempt in range(3):
    try:
        return validate(json.loads(raw))
    except (json.JSONDecodeError, ValueError) as e:
        messages.append({"role": "assistant", "content": raw})
        messages.append({"role": "user",
            "content": f"That failed: {e}. Return ONLY valid JSON."})
        raw = call(messages)    # try again with the error in context
```

Cap the retries (cost + latency). Schema-constrained output makes repairs rare — but a loop is your safety net when it isn't available.

A catalog of mechanical repairs

Symptom (before)	Fix	Safe?
`` ```json\n{...}\n``` ````	strip fences	yes
`{"a": 1,}` trailing comma	regex before `}` / `]`	yes
`Sure! {"a": 1}` prose	first `{` ... last `}`	yes
`{...}{...}` two objects	brace-counter (slide 14b)	care
`{“t”: “hi”}` curly quotes	`str.translate` → ASCII	yes
`{'a': 1}` single quotes	— use the re-ask loop	no

Each rule must be narrow enough it can't corrupt a valid string. When it isn't safely mechanical, return `None` and let the loop handle it.

Cap the loop — and plan for failure

- Cap the retries. Each try costs tokens and latency; a model wrong twice may loop forever. Three is sane.
- Include the bad reply. Append the model's wrong `assistant` turn before your correction — it sees its mistake next to the error.
- When repair is exhausted, don't crash. Catch it and pick a deliberate fallback:

```
try:
    order = get_order(messages)
except RuntimeError:
    order = None    # a default, hand off to a human, or a friendly error
```

Which fallback is a product decision. The rule: never let unparseable output silently become bad data downstream.

Which failure → which fix

Classify first, then pick the cheapest fix that applies.

What you observe	Class	Fix
fence, preamble, trailing comma, curly quotes	parse	local repair
two objects / stray `}` in prose	parse	brace-counter
single quotes `{'a': 1}`	parse (ambiguous)	re-ask loop
missing key / wrong type	shape	validate → loop
`qty: -3`, out of range	semantic	validate → reject/loop
plausible-but-wrong value	semantic	no parser saves you

Mechanical → fix locally. Shape/semantic → feed the error to the loop. A helper that tries all three corrupts good input.

PREDICT → RUN → MODIFY

Will the repair loop save us?

We seed a deliberately broken reply:

```
{'city': 'Bangkok', temp_c: 34,}
```

Predict:

- On which attempt (1 / 2 / 3) does it first print OK?
- What is the exact attempt-1 error? (single quotes? bare key? trailing comma?)

After the reveal: lower `max_tries` to 1 and watch it give up.

Repair Demo

```
# A repair loop you can watch work. We seed a deliberately BROKEN reply, then let
# the model fix it when we feed the parse error back. Pure stdlib parsing.
import json
from openai import OpenAI

client = OpenAI()


def extract_json(text):
    # strip a ```json ... ``` fence (the model often wraps its fix in one) so the
    # repaired reply actually parses; then keep first { .. last }
    t = text.strip()
    if t.startswith("```"):
        t = t.split("```")[1]
        if t.lstrip().startswith("json"):
            t = t.lstrip()[4:]
        t = t.strip()
```

Repair Json Local

```
# LOCAL repair: no model call. The #2 real malformation after fences is a trailing
# comma — valid Python, invalid JSON. Fix it in-process; fall back to None when the
# string isn't safely patchable. (HW1 Q3 is exactly this.) Ctrl/Cmd+Enter.
import json
import re


def extract_json(text):
    t = text.strip()
    if t.startswith("```"):
        t = t.split("```")[1]
        if t.lstrip().startswith("json"):
            t = t.lstrip()[4:]
        t = t.strip()
    s, e = t.find("{"), t.rfind("}")
    return t[s:e + 1] if s != -1 and e != -1 else t
```

Brace Counter

```
# Hardened extraction: walk the string tracking brace DEPTH (and string state) to
# return the FIRST balanced object. Beats `first { .. last }` when there's a stray
# `}` in trailing prose or a SECOND object. No model call. Ctrl/Cmd+Enter.
import json


def first_object(text):
    start = text.find("{")
    if start == -1:
        return None
    depth, in_str, esc = 0, False, False
    for i in range(start, len(text)):
        ch = text[i]
        if in_str:
            if esc:
                esc = False
            elif ch == "\\":
                esc = True
            elif ch == '"':
```

repair_json's job is to delete a trailing comma that sits right before a closing } or]. The model returns the JSON object with note set to the text "a, b," plus a stray trailing comma after the last field. What should repair_json give back?

- Nothing (None) — the commas inside the note text break it
- **The object with note still "a, b," and the n field — only the trailing comma before } is removed, commas inside the text are left alone ✓**
- The object but with every comma stripped, so note becomes "a b"
- An error — it can't tell trailing commas from in-text commas

Richer schemas: nested · enum · Optional

Past `{item, qty, price}`: Pydantic checks all three in one call.

```
class Size(str, Enum):
    small = "small"; large = "large"   # closed set
class Customer(BaseModel):
    name: str
    phone: str | None = None           # Optional: absent or null
class Order(BaseModel):
    item: str; qty: int; size: Size  # size must be in the enum
    customer: Customer               # nested, validated recursively
```

- Enum rejects `"venti"` for free.
- Optional = "absent or null", *not* "any type".

Proxy still won't enforce it (no `response_format`) — validate-and-repair makes it stick.

A Pydantic field is declared `phone: str | None = None`. The model returns {"name": "Mai", "phone": 42}. What does model_validate_json do?

- Accepts it — Optional means the field skips validation
- Accepts it — None default means any value is allowed
- **Rejects it — Optional means 'absent or null', but a present phone must still be a string ✓**
- Coerces 42 to the string "42" silently

The model can read images too

A vision-language model takes an image in the message — point it at a receipt and ask for JSON. `gemma-4-E4B-it` is multimodal.

```
content = [
  {"type": "text",
   "text": "Extract invoice_number, date, total."},
  {"type": "image_url",
   "image_url": {"url": "..."}},
]
```

The reply is still text — same extract → parse → validate → repair pipeline. Live: the 📎 demo / `extract_from_image` on rag-demo.nat-d.uk.

The API returns syntactically valid JSON, but the `price` field is missing. Which layer should have caught this?

- JSON mode (json_object) — it guarantees the output
- json.loads — it would have raised on the missing key
- **Schema / field validation ✓**
- The repair loop alone — no validation needed

Where this goes: tool calling

A tool call is just structured output where the schema *is* the function's parameters.

- You describe a tool's arguments with the same JSON Schema.
- The model emits arguments; you parse + validate them — exactly today's pattern.
- Master structured output here and tool calling (Lecture 10) is a small step.

Define once, reuse

A Pydantic `Order` becomes the tool schema, the validator, and — from Lecture 3 — your API request body. One shape, the whole pipeline.

Structured output that holds up

- Constrain first, prompt second. Use a schema / JSON mode when you have it; the prompt is the fallback.
- Always validate on your side. Never trust the model's keys or types — `json.loads` then check.
- Repair, don't crash. Feed the parse error back, with a hard retry cap.
- temperature 0 for extraction — same input, same JSON.
- Define the shape once (a Pydantic model) and reuse it for the schema and the validator.

> Treat the LLM as an untrusted input source: parse defensively, exactly like form data.

Your proxy does NOT support schema-constrained output, only plain chat. To get reliable typed orders out of the model (the HW1 extractor), what's the right design?

- Prompt for JSON with an example + temperature 0, then json.loads + validate, then repair once on failure ✓
- Raise temperature so the model is more creative and fixes itself
- Trust json.loads — if it parses, the data is correct
- Skip validation; the prompt says 'ONLY JSON', so the output is guaranteed

Recap & what's next

You can now

- Say why "just parse the text" breaks
- Prompt for JSON — and know why JSON mode / schemas don't help on our proxy
- Validate with `json.loads` + a check
- Repair bad output by feeding back the error

Next lecture

Lecture 3

Backend & REST API — serve this extraction behind an endpoint your frontend can call.

Builds on Lecture 1; the validated-JSON habit is used everywhere after. Read the notes, then carry it into HW1.